

**COMPUTER VISION – CO3
ASSESSMENT TOOL 2
10-MARK ANSWERS**

SIMATS ENGINEERING | Course Code: ITA0515 | Slot B

**Name: Sriram R
Register No: 192525198**

1. Preprocessing and Feature Detection

Introduction

In computer vision, feature detection is used to identify important structures in an image such as edges, corners, textures, and key points. Images captured under different lighting conditions may have variations in brightness, contrast, and shadows. These variations can cause feature detectors to produce inconsistent results. Therefore, preprocessing is an important step before feature detection.

Problem Due to Varying Lighting

When the same object is captured under different lighting conditions, its pixel intensity values may change significantly. Bright lighting can increase pixel intensity, low lighting can make regions dark, shadows can hide features, and uneven illumination can create unwanted edges. Consequently, a feature detector may detect a feature in one image but fail to detect the same feature in another image.

Role of Preprocessing

1. Grayscale Conversion: Converts RGB images into a single intensity channel, simplifying processing and reducing computation.
2. Noise Reduction: Gaussian or median filtering reduces noise that can create false edges and unwanted key points.
3. Contrast Enhancement: Histogram equalization improves the difference between image regions. Adaptive methods can help with uneven illumination.
4. Normalization: Scales intensity values to a common range, making images captured under different brightness levels more comparable.
5. Illumination Correction: Reduces the effect of shadows and uneven lighting.

Effect on Feature Detection

After preprocessing, noise-related false features are reduced, important edges become clearer, corners can be detected more consistently, and feature descriptors become more stable. Matching between images therefore becomes more reliable.

Example

If the same object is captured in bright sunlight and low light, a detector may identify different corners. If the images are converted to grayscale, denoised, contrast-enhanced, and normalized first, important structural features become more consistent.

Conclusion

Preprocessing is essential for reliable feature detection under varying lighting conditions. Grayscale conversion, noise reduction, contrast enhancement, normalization, and illumination correction reduce unwanted variations and improve the consistency and accuracy of feature detection.

2. Edge vs Corner Detection

Introduction

Edge detection and corner detection are important techniques in computer vision. Edge detection identifies boundaries where image intensity changes significantly, whereas corner detection identifies points where intensity changes in multiple directions. In this case, edge detection fails to provide suitable key points for matching.

Why Edge Detection Is Insufficient

1. Edges are not unique points: An edge is usually a line or continuous boundary rather than a specific point.
2. Poor localization: Many pixels along a straight edge can have similar characteristics.
3. Ambiguity: Similar edges may be difficult to correspond between images.
4. Sensitivity to transformations: Changes in viewpoint, scale, and illumination can affect detected edges.
5. Textured regions: Edge detection may produce many boundaries without identifying the most distinctive matching points.

Importance of Corner Detection

A corner is a point where image intensity changes significantly in more than one direction. For example, the corner of a square has strong intensity variation horizontally and vertically. Corners are therefore localized and distinctive key points.

Common corner detectors include Harris Corner Detector, Shi-Tomasi Corner Detector, and FAST.

Harris Corner Detection

The Harris method examines intensity changes around a pixel in different directions. A flat region has little change, an edge has significant change mainly in one direction, and a corner has significant change in multiple directions.

Edge vs Corner

Edge detection mainly identifies boundaries and is less suitable for selecting unique matching points. Corner detection identifies distinctive localized points and is more suitable for feature matching.

Application

The improved workflow is:

Input Image → Preprocessing → Corner Detection → Key Points → Feature Description → Feature Matching.

Conclusion

Edge detection alone is insufficient for reliable feature matching because edges are continuous structures and may not provide unique key points. Corner detectors such as Harris, Shi-Tomasi, or FAST are more appropriate for extracting stable key points.

3. Hough Transform in Noisy Images

Introduction

The Hough Transform is a feature detection technique used to detect geometric structures such as lines and circles. It transforms image points into a parameter space and identifies peaks representing possible geometric shapes. Noise can make these results inaccurate.

Effect of Noise

Noise introduces unwanted intensity variations. When an edge detector is applied to a noisy image, noise may be incorrectly detected as edges. These false edges are then supplied to the Hough Transform, producing false lines, unclear accumulator peaks, and inaccurate line positions.

Preprocessing Improvements

1. Gaussian Filtering: Smooths the image and reduces random noise.
2. Median Filtering: Reduces certain types of noise while preserving important boundaries.
3. Edge Detection: After smoothing, an edge detector such as Canny can produce a cleaner edge map.

Parameter Selection

1. Threshold: A low threshold may produce many false lines, while a high threshold may miss weak genuine lines. An appropriate threshold is required.
2. Parameter-space resolution: Coarse resolution may reduce localization accuracy, while very fine resolution can increase computational cost.
3. Minimum Line Length: In probabilistic Hough Transform, increasing minimum line length can remove short false lines caused by noise.
4. Maximum Line Gap: A suitable gap allows broken portions of a real line to be connected without joining unrelated structures.

Improved System

Noisy Image → Gaussian/Median Filtering → Edge Detection → Clean Edge Map → Hough Transform → Threshold and Parameter Optimization → Accurate Line Detection.

Example

In a road image affected by camera noise, direct Hough Transform may detect many false lines. Smoothing followed by clean edge detection reduces false points and improves line detection.

Conclusion

Noise significantly affects Hough Transform because false edges can create false accumulator peaks. Noise reduction, proper edge detection, and suitable parameter selection greatly improve the accuracy of Hough-based line detection.

4. Feature Matching Under Transformations

Introduction

Feature matching identifies corresponding points between two images. Traditional feature matching may work when images have similar scale and orientation, but it can fail when the same object is captured at different scales or orientations. Scale-invariant methods are designed to handle these changes.

Problem With Traditional Feature Matching

When the same object is captured from different distances, its apparent size changes. Rotation also changes the orientation of local structures. Direct comparison of image regions may therefore fail because the pixel locations and feature appearance are different.

Scale Variation

A detector that works at one image scale may not detect the same feature at another scale. Scale-invariant methods search for features across multiple scales so that the same important structure can be detected even when its apparent size changes.

Rotation Variation

Rotation changes the orientation of image features. Traditional descriptors can change after rotation. Scale-invariant methods can assign an orientation to detected features and construct descriptors relative to that orientation, making the representation more resistant to rotation.

Example: SIFT

SIFT (Scale-Invariant Feature Transform) performs scale-space construction, keypoint detection and localization, orientation assignment, descriptor generation, and feature matching. These steps help identify distinctive features despite changes in scale and orientation.

Advantages

- Better matching under different scales.
- Better matching under rotation.
- More stable key points.
- More distinctive feature descriptions.
- Improved correspondence between images.
- Greater robustness to common transformations.

Comparison

Traditional matching is sensitive to scale and rotation, whereas scale-invariant matching is designed to remain more stable under these changes.

Application

For two images containing the same object at different scales and orientations, scale-invariant methods identify stable local structures and generate descriptors that are less dependent on scale and orientation. Therefore, corresponding features can still be matched successfully.

Conclusion

Scale-invariant feature matching performs better because it is designed to handle scale and orientation changes. Methods such as SIFT provide stable key points and descriptors, making them suitable for matching images captured under different transformations.

5. PCA for Feature Optimization

Introduction

Feature extraction in computer vision can generate a large number of features. Some may be redundant or highly correlated. Processing all of them increases computational cost, memory requirements, and processing time. Principal Component Analysis (PCA) reduces dimensionality while retaining the major information in the data.

Problem With High-Dimensional Features

A large feature set can cause high computational cost, increased memory usage, longer processing time, redundant information, and reduced efficiency. Correlated features may contain similar information and do not all need to be processed independently.

What Is PCA?

PCA transforms the original features into a new coordinate system. The new variables are called principal components. The first principal component represents the maximum variance, the second represents the next highest variance while being uncorrelated with the first, and so on.

Steps in PCA

1. Collect the feature data into a feature matrix.
2. Standardize features when necessary so scale differences do not dominate.
3. Calculate the covariance matrix to represent relationships between features.
4. Calculate eigenvalues and eigenvectors. Eigenvectors give principal directions and eigenvalues indicate the variance represented.
5. Select components with larger eigenvalues.
6. Project the original data onto the selected components.

Example

Suppose a system extracts 100 features from an image. PCA may represent most of the important variation using only 20 principal components:

100 Features → PCA → 20 Important Components.

This reduces the dimensionality by 80%.

Benefits

1. Reduced computational cost.
2. Reduced memory usage.
3. Reduced redundancy.
4. Faster processing.
5. Potential reduction of less informative variation.
6. Improved efficiency for large datasets.

Applications

PCA can be used before feature matching, image classification, object recognition, face recognition, and image retrieval.

Important Consideration

PCA does not simply select existing features. It creates new components that are combinations of the original features. The number of components must be selected carefully so that dimensionality is reduced without losing too much important information.

Conclusion

PCA is an effective technique for optimizing high-dimensional image features. It reduces redundancy by transforming correlated features into fewer principal components, decreasing computational cost, memory usage, and processing time while retaining major information.